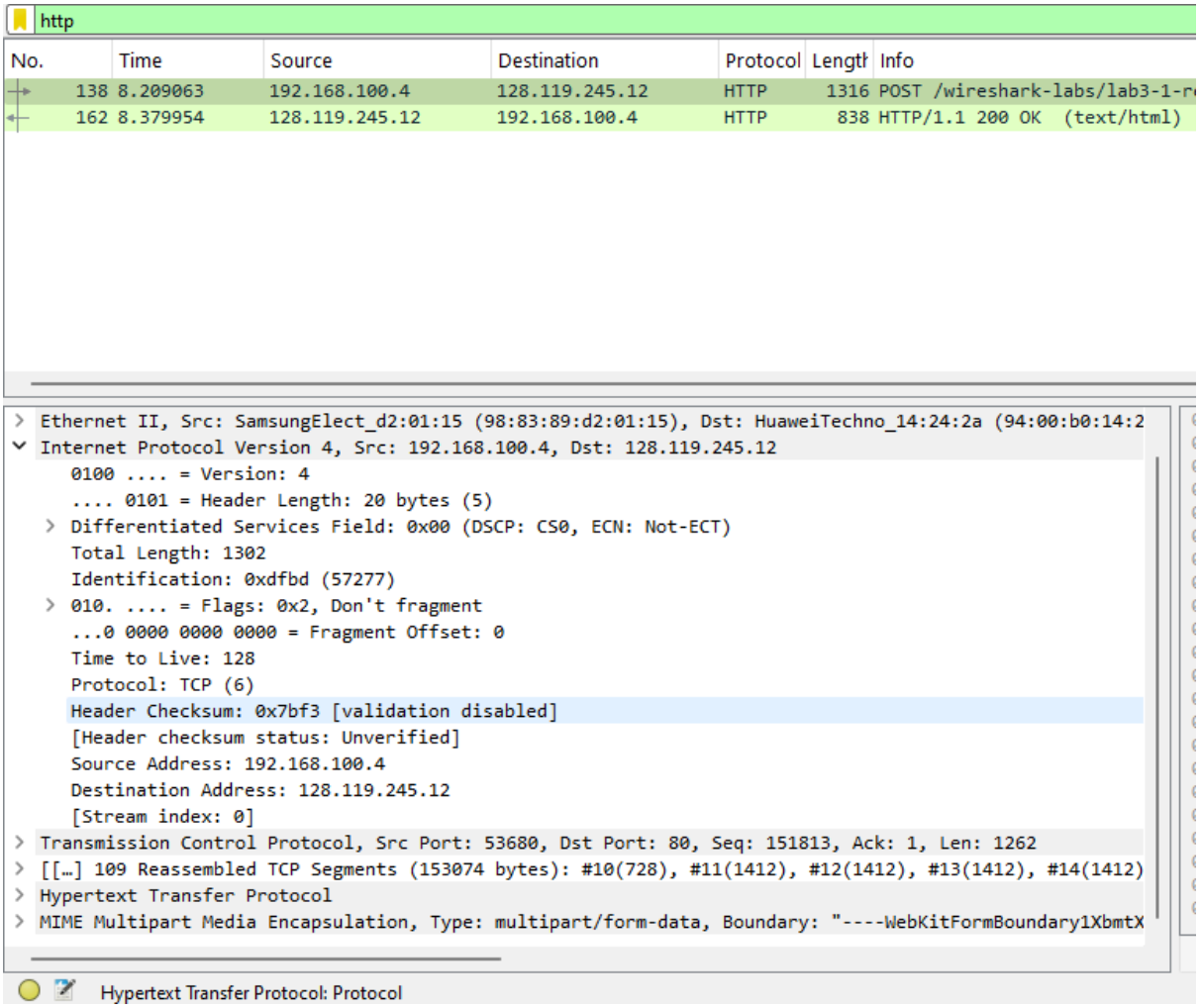
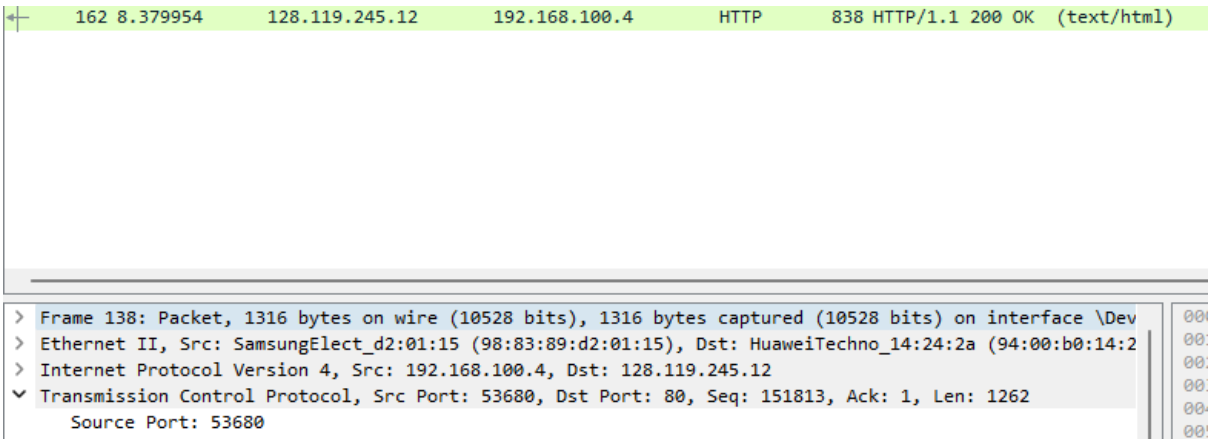
# Leonardo H. C. Oliveira - 12311BCC003

**Identificação IPv4:**




**01 - What is the IP address and TCP port number used by the client computer(source) that is transferring the alice.txt file to gaia.cs.umass.edu?**

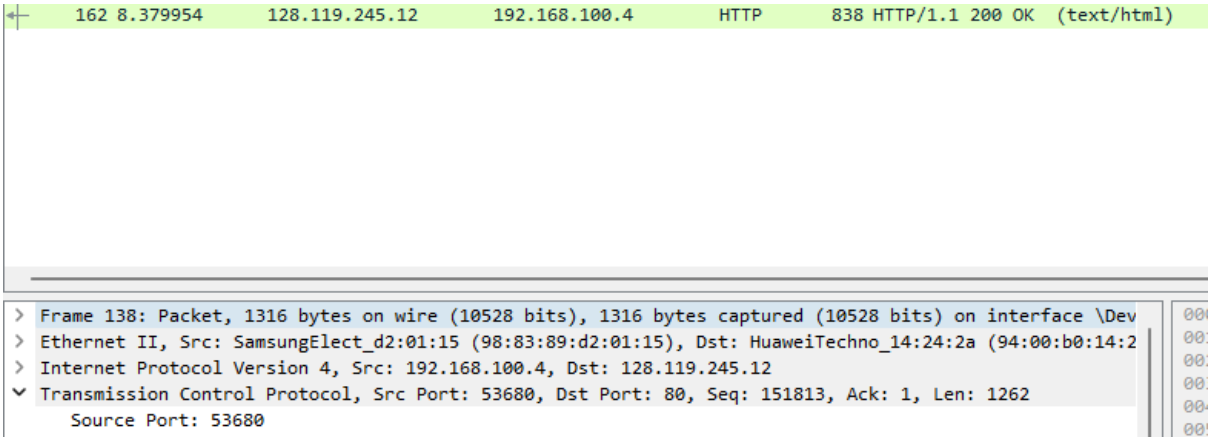
O endereço IP do servidor é 128.119.245.12 e o número da porta TCP é 80.




**02 - What is the IP address of gaia.cs.umass.edu? On what port number is it sending and receiving TCP segments for this connection?**

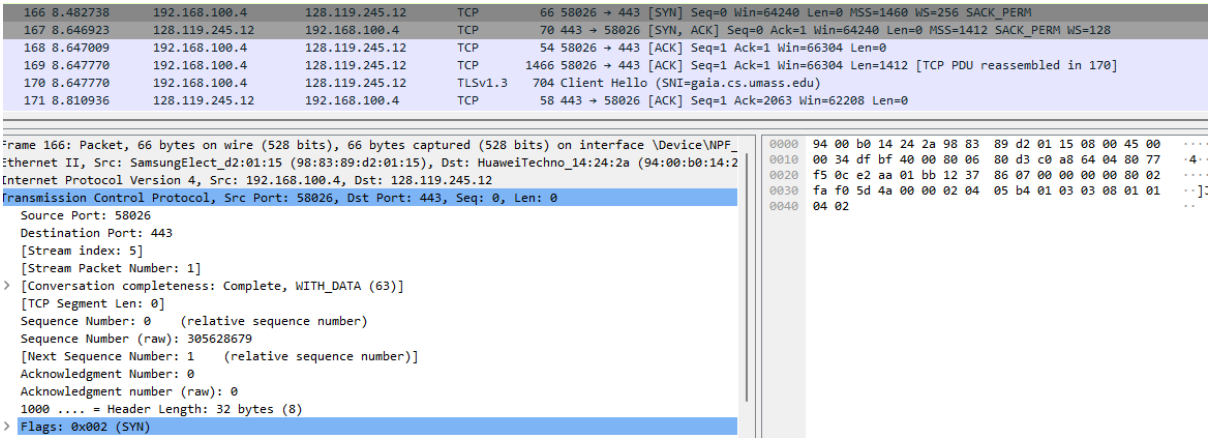
.

O endereço IP do cliente é 192.168.100.4 e o número da porta TCP é 53680.



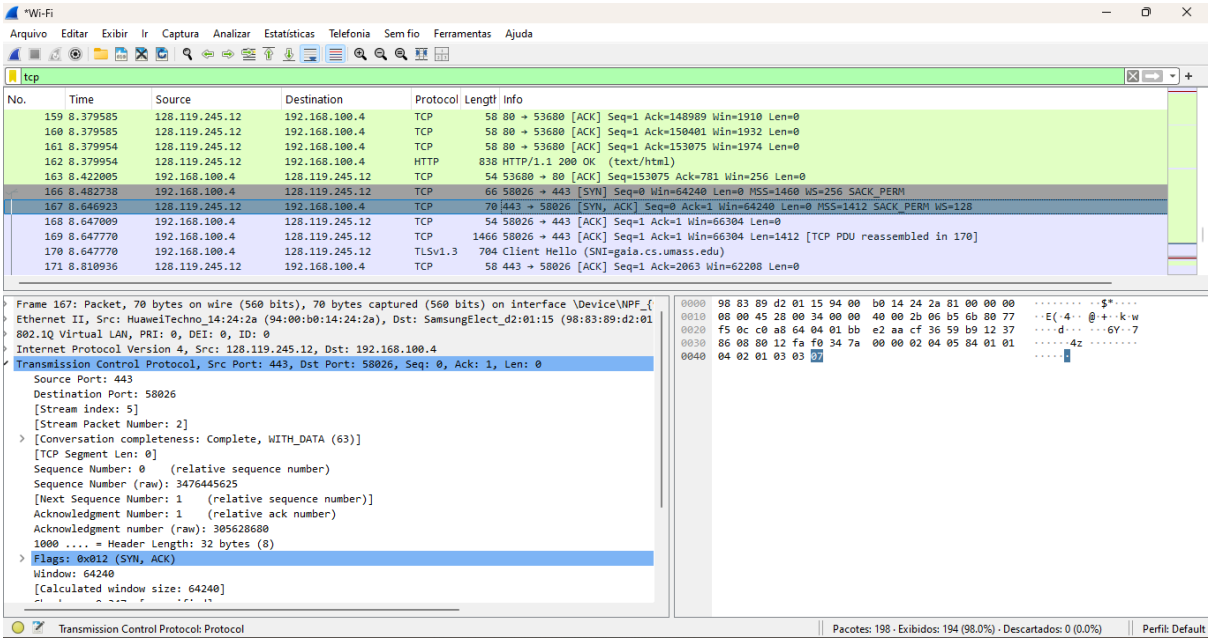

03 - **What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu? What is it in this TCP segment that identifies the segment as a SYN segment ? Will the TCP receiver in this session be able to use Selective Acknowledgments ?**

O número de sequência raw é 305628679. O que identifica este segmento é a Flag SYN estar ativada. E sim, como é mostrado pela presença de SACK_PERM.



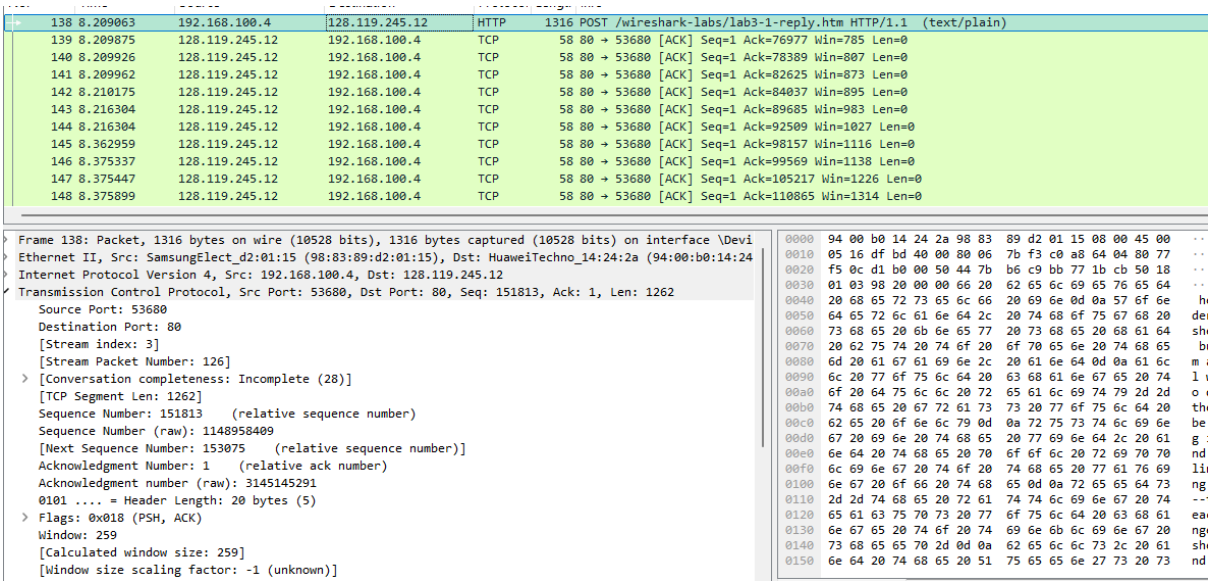
**04 - What is the sequence number of the SYNACK segment sent by gaia.cs.umass.edu to the client computer in reply to the SYN? What is it in the segment that identifies the segment as a SYNACK segment? What is the value of the Acknowledgement field in the SYNACK segment? How did gaia.cs.umass.edu determine that value?**

O número de sequência do segmento SYNACK enviado pelo servidor é 3476445625. O segmento é identificado como SYNACK pela presença das flags SYN e ACK ativadas simultaneamente (0x012) no cabeçalho TCP. O valor do campo Acknowledgement neste segmento é 305628680, o qual foi determinado pelo servidor gaia.cs.umass.edu somando 1 ao número de sequência inicial do segmento SYN recebido anteriormente do computador cliente.




**05 - What is the sequence number of the TCP segment containing the header of the HTTP POST command? How many bytes of data are contained in the payload (data) field of this TCP segment? Did all of the data in the transferred file alice.txt fit into this single segment?**

O número de sequência do segmento TCP é 151813. O campo de dados contém 1262 bytes. Não, o que é evidenciado pela indicação de 109 Reassembled TCP Segments.




**06 - Consider the TCP segment containing the HTTP "POST" as the first segment in the data transfer part of the TCP connection.**

**• At what time was the first segment (the one containing the HTTP POST) in the data-transfer part of the TCP connection sent?**

**• At what time was the ACK for this first data-containing segment received?**

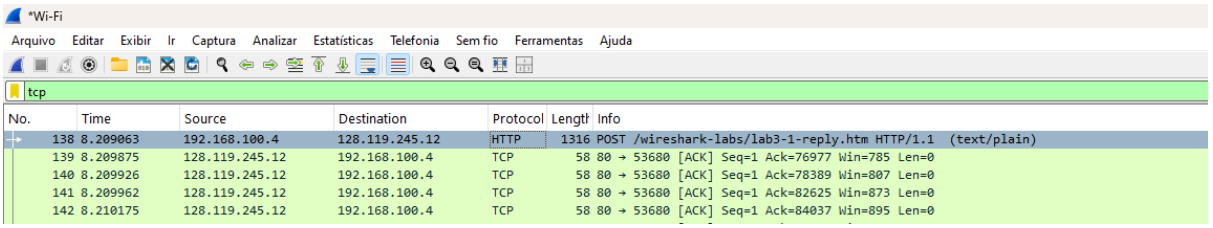
**• What is the RTT for this first data-containing segment?**

**• What is the RTT value the second data-carrying TCP segment and its ACK?**

**• What is the EstimatedRTT value (see Section 3.5.3, in the text) after the ACK for the second data-carrying segment is received? Assume that in making this calculation after the received of the ACK for the second segment, that the initial value of EstimatedRTT is equal to the measured RTT for the first segment, and then is computed using the EstimatedRTT equation on page 242, and a value of a = 0.125.**

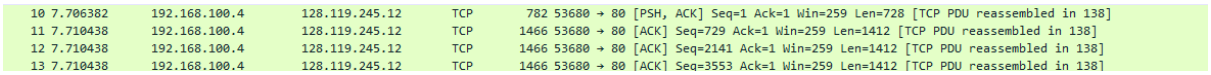
**Note: Wireshark has a nice feature that allows you to plot the RTT for each of the TCP segments sent. Select a TCP segment in the "listing of captured packets" window that is being sent from the client to the gaia.cs.umass.edu server. Then select: Statistics->TCP Stream Graph->Round Trip Time Graph.**

O primeiro segmento foi enviado no tempo 7.706382. O ACK foi recebido no tempo 7.736382, resultando em um RTT de 0.030. O RTT para o segundo segmento de dados foi de 0.040 segundos. O valor do EstimatedRTT é de 0.03125.



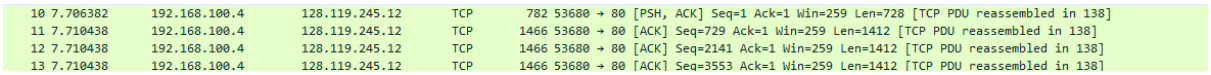
**07 - What is the length (header plus payload) of each of the first four data-carrying TCP segments?**

O comprimento total do primeiro segmento TCP é de 748 bytes(20 bytes de cabeçalho + 728 bytes de carga útil). Os comprimentos do segundo, terceiro e quarto segmentos são idênticos, sendo 1432 bytes cada(20 bytes de cabeçalho + 1412 bytes de carga útil).



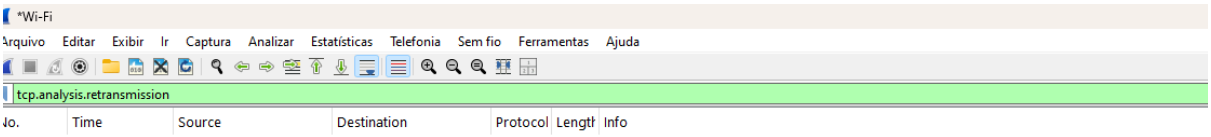

**08 -What is the minimum amount of available buffer space advertised to the client by gaia.cs.umass.edu among these first four data-carrying TCP segments? Does the lack of receiver buffer space ever throttle the sender for these first four datacarrying segments?**

O mínimo de quantidade disponível de buffer é de 259 bytes, e o servidor nunca limita o remetente.




**09 - Are there any retransmitted segments in the trace file? What did you check for (in the trace) in order to answer this question?**

Não houve segmentos retransmitidos. Eu apliquei o filtro de retransmissão.




**10 - How much data does the receiver typically acknowledge in an ACK among the first ten datacarrying segments sent from the client to gaia.cs.umass.edu? Can you identify cases where the receiver is ACKing every other received segment (see Table 3.2 in the text) among these first ten data-carrying segments?**

O servidor reconhece segmentos de 1412 bytes. Sim, os pacotes 19 e 22 são um exemplo disso.

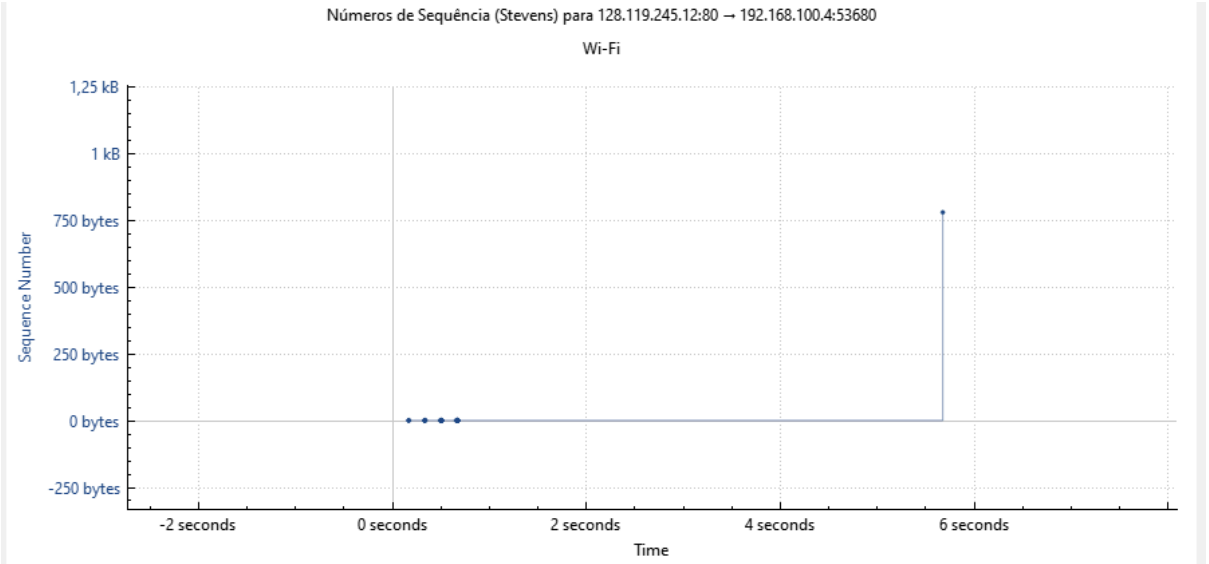
| | | | | | |
|---|---|---|---|---|---|
| 20 | 7.872672 | 128.119.245.12 | 192.168.100.4 | TCP | 58 80 → 53680 [ACK] Seq=1 Ack=729 Win=497 Len=0 |
| 21 | 7.872751 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=13437 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 22 | 7.875808 | 128.119.245.12 | 192.168.100.4 | TCP | 58 80 → 53680 [ACK] Seq=1 Ack=7789 Win=611 Len=0 |
| 23 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=14849 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 24 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [PSH, ACK] Seq=16261 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 25 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=17673 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 26 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=19085 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 27 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=20497 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 28 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=21909 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 29 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=23321 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 30 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=24733 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 31 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=26145 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 32 | 7.875897 | 192.168.100.4 | 128.119.245.12 | TCP | 1466 53680 → 80 [ACK] Seq=27557 Ack=1 Win=259 Len=1412 [TCP PDU reassembled in 138] |
| 33 | 7.876331 | 128.119.245.12 | 192.168.100.4 | TCP | 58 80 → 53680 [ACK] Seq=1 Ack=13437 Win=611 Len=0 |

**11 - What is the throughput (bytes transferred per unit time) for the TCP connection? Explain how you calculated this value**.

O throughput  é de 227.256 bytes/segundo. Esse valor foi calculado dividindo-se o tamanho total da carga útil transferida (153.074 bytes) pelo tempo total de transmissão (0,673572 segundos).

**12 - Use the Time-Sequence-Graph(Stevens) plotting tool to view the sequence number versus time plot of segments being sent from the client to the gaia.cs.umass.edu server. Consider the “fleets” of packets sent around t = 0.025, t = 0.053, t = 0.082 and t = 0.1. Comment on whether this looks as if TCP is in its slow start phase, congestion avoidance phase or some other phase. Figure 6 shows a slightly different view of this data**

A análise do gráfico revela que a conexão TCP está na fase de Slow Start (Partida Lenta). Isso é evidenciado pelo comportamento das frotas de pacotes nos intervalos de tempo observados, que aumentam de tamanho exponencialmente, diferentemente do crescimento linear que caracterizaria a fase de prevenção de congestionamento.



**13.These “fleets” of segments appear to have some periodicity.What can you say about the period?**

A periodicidade observada entre as frotas de segmentos corresponde ao RTT (Round Trip Time) da conexão. Isso ocorre porque, na fase de partida lenta, o remetente dispara uma janela de pacotes e precisa aguardar o retorno dos ACKs antes de enviar o próximo lote.